After User Abstraction: Backend Retained Demand and Just-in-Time Liquidity in Institutionally Connected Digital Assets

S. Meta

April 2026

Abstract

Paper 1 defined retained demand by type. This paper extends that analysis by asking where retained demand is located after user-facing token exposure is abstracted away.[1] It shifts the audit question from what forms of holding matter to whose balance sheet or operational system carries that holding.

The paper argues that user abstraction changes the location of demand observation, but does not by itself create retained demand. Gasless user experiences, embedded wallets, paymasters, custodial interfaces, platform balances, and treasury automation may remove token exposure from the user interface. Yet this disappearance does not imply that the underlying asset has become economically irrelevant. Nor does it imply that backend operators must necessarily hold the asset. The decisive question is whether wallets, exchanges, custodians, paymasters, market makers, treasury operators, clearing systems, or other infrastructure-layer actors must pre-position, reserve, collateralize, inventory, or buffer the asset in order for the relevant workflow to function.

The central distinction developed in the paper is between pre-positioned inventory and just-in-time liquidity. If an asset can be procured only when needed, used immediately, and released without meaningful timing risk, market impact, counterparty risk, collateral lockup, or operational fragility, then retained demand remains weak. Use without inventory is weak retained demand. Backend retained demand becomes more likely when the expected cost and risk of just-in-time procurement exceed the cost and friction of maintaining pre-positioned inventory.

The paper contributes three linked elements. Section 3 defines backend retained demand as retained demand located on the balance sheets or operational systems of infrastructure-layer actors rather than end users. Section 4 treats just-in-time liquidity as a falsification condition for retained-demand claims. Section 6 and Appendix A provide a cross-asset audit map and worksheet for comparing BTC, ETH/SOL, LINK, stablecoins and tokenized deposits, XRP, and RWA-specific assets according to backend holder type, retained-demand form, JIT risk, operator friction, evidence status, and key falsifiers.

The paper does not offer a price thesis, investment thesis, or asset-specific advocacy argument. XRP appears only as one cross-asset case among others. The broader contribution is methodological and operational: retained demand after user abstraction should be audited at the infrastructure balance-sheet layer, not at the user interface.

1. Introduction: From User Exposure to Backend Balance Sheets

Digital asset usage is increasingly becoming invisible to the user. Wallets abstract private-key management. Applications sponsor transaction fees. Paymasters allow transactions to occur without users directly holding the native gas asset. Exchanges, custodians, and embedded finance platforms present account balances and service interfaces rather than raw token mechanics. Treasury systems and institutional workflows may route, settle, hedge, or rebalance digital assets without end users observing the underlying asset movements at all.

This development creates a new analytical problem. Earlier digital asset analysis often treated user holding as a primary observable signal of demand. If users held the token, the asset appeared relevant; if they did not, the

asset appeared marginal. But in a more abstracted infrastructure environment, this inference becomes unreliable. Token exposure may disappear from the user interface even while the asset remains operationally relevant somewhere else in the system.

At the same time, the opposite inference is equally unsafe. The fact that users no longer see or hold a token does not prove that backend retained demand exists. A system may abstract the token because a platform, paymaster, market maker, custodian, or treasury operator holds inventory on behalf of users. But it may also abstract the token because the system can procure the asset just in time, route through substitutes, net exposures internally, settle through stablecoins or tokenized deposits, or avoid asset-specific holdings altogether.

The question therefore changes. The relevant question is no longer simply whether users hold the asset. The question is whether someone else must hold it for the system to work.

Paper 1 classified retained demand by form. This paper classifies retained demand by location. The question is no longer only whether demand appears as inventory, collateral, liquidity buffer, or waiting liquidity, but whose balance sheet or operational system carries that demand after user-facing exposure has been abstracted away.

This shift matters because user abstraction can produce several different outcomes. In one case, demand relocates from the user to an infrastructure-layer actor. A paymaster may hold gas inventory. A market maker may retain execution inventory. A custodian may hold collateral-related balances. A treasury operator may maintain liquidity buffers. A clearing or settlement layer may require waiting liquidity. In another case, the same user-facing abstraction may produce little or no retained demand because the asset can be obtained when needed, used briefly, and released.

The distinction between these outcomes is the core of the paper. A token can be used without being meaningfully retained. A backend operator can support an asset without holding material inventory. A workflow can be token-enabled without becoming balance-sheet dependent on the token. For that reason, the paper treats just-in-time liquidity not as a technical detail, but as a falsification condition. If just-in-time procurement is sufficient, backend retained demand remains weak.

The central proposition is straightforward: backend retained demand becomes more likely when the expected cost and risk of just-in-time procurement exceed the cost and friction of maintaining pre-positioned inventory. Such conditions may arise when execution windows are short, liquidity is fragmented, market impact is material, counterparties may disappear, stress conditions impair procurement, collateral must be locked in advance, or operational reliability requires pre-funded balances. Conversely, if these conditions are absent, usage may scale while retained demand remains shallow.

The contribution of this paper is threefold. First, it introduces backend retained demand as a location-based extension of retained-demand analysis. Second, it develops the distinction between pre-positioned inventory and just-in-time liquidity as the central audit test for user-abstracted digital asset workflows. Third, it applies the framework across asset categories, including BTC, ETH/SOL, LINK, stablecoins and tokenized deposits, XRP, and RWA-specific assets, without treating any one asset as the default beneficiary of infrastructure growth.

Within the broader retained-demand audit series, this paper functions as the sixth layer: the backend-location layer. Paper 1 defined retained demand by type; Paper 2 examined closure competition; Paper 3 translated retained demand into required liquidity density; Paper 4 separated compression, bypass, and amplification; and Paper 5 gated sizing by evidence. The present paper extends that sequence by asking where retained demand is carried after user-facing token exposure disappears.

This paper can be read independently as an audit of backend retained demand after user abstraction. However, its full meaning depends on the preceding retained-demand audit sequence: Paper 1 defines retained demand by type, Paper 2 distinguishes expansion from closure, Paper 3 separates liquidity-density sizing from price

prediction, Paper 4 classifies compression, bypass, and amplification, and Paper 5 establishes evidence gates before sizing.

The paper is not a price thesis, investment thesis, or asset advocacy document. It does not argue that user abstraction necessarily strengthens digital asset demand, nor that it necessarily weakens it. Its purpose is to identify where retained demand would appear if it exists, what conditions would make just-in-time procurement insufficient, and what evidence would falsify backend retained-demand claims.

The remainder of the paper proceeds as follows. Section 2 explains user abstraction and the relocation of demand observation. Section 3 defines backend retained demand and classifies possible backend holders. Section 4 develops the distinction between pre-positioned inventory and just-in-time liquidity. Section 5 examines operator friction and multi-asset management. Section 6 applies the framework across major asset categories. Section 7 defines falsification conditions. Section 8 concludes. Appendix A provides a backend retained-demand audit template.

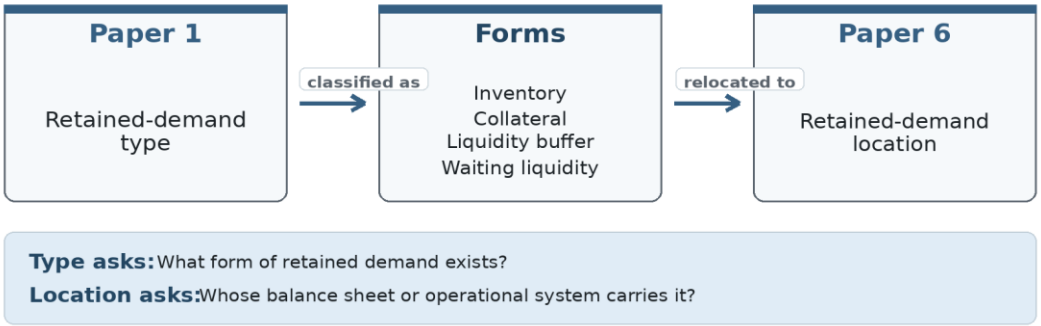

Figure 1. The shift from retained-demand type to retained-demand location.

2. User Abstraction and the Relocation of Demand

User abstraction refers to the removal, concealment, or delegation of asset-specific exposure from the user-facing layer of a digital asset workflow. A user may initiate a transaction, make a payment, interact with an application, receive a tokenized asset, or access a financial service without directly holding, selecting, acquiring, or even noticing the asset that enables part of the underlying process.

Paymasters and account-abstraction designs provide a concrete example: a user may transact without holding the native gas asset because a backend actor sponsors or manages the required fee exposure.[2] Similar abstraction may occur through embedded wallets, custodians, internalized platform balances, automated treasury systems, and settlement providers. The important analytical point is not the user-interface method, but the relocation of the holding question.

The immediate implication is that visible user holding becomes a weaker indicator of retained demand. A decline in direct user exposure does not necessarily imply that the asset has lost operational relevance. In some cases, exposure has simply moved to another actor. In other cases, however, abstraction reduces or eliminates the need for asset-specific holding because the backend can obtain the asset only when needed, use it momentarily, and then release or hedge the exposure without material cost or risk.

Relocation should therefore not be assumed. The same user-facing experience can imply very different retained-demand structures. A gasless user experience may require a paymaster to maintain gas inventory, or it may rely on dynamic replenishment. A custodial platform may maintain operational balances for withdrawals, settlement, routing, or market-making, or it may minimize holdings through internal netting, omnibus custody, external liquidity providers, or just-in-time execution.

In institutional environments, user abstraction becomes even more complex. A corporate treasury department may not manually hold a digital asset for every workflow. Instead, a treasury platform, custodian, broker, market maker, or settlement provider may manage the relevant liquidity. Tokenized asset platforms may allow users to interact with tokenized funds, securities, or claims without directly managing the settlement asset. Payment providers may allow merchants or counterparties to receive fiat, stablecoins, or account balances while another layer handles conversion and settlement.

These examples show why the disappearance of token exposure from the user interface is analytically ambiguous. It can mean that the asset has become less relevant. It can also mean that the asset has become more deeply embedded in the backend. The difference depends on whether any infrastructure-layer actor must retain the asset in inventory, collateral, liquidity-buffer, or waiting-liquidity form.

The analytical task is therefore to relocate the audit point. Instead of asking only whether users hold the asset, the analyst must ask whether the asset is held elsewhere, by whom, for what purpose, for how long, under what constraints, and with what consequences if it is removed. User abstraction does not end retained-demand analysis. It makes retained-demand analysis more balance-sheet specific.

This paper uses the term backend retained demand to describe retained demand located at the infrastructure layer rather than the user layer. It may appear on the balance sheets or operational systems of wallets, exchanges, custodians, paymasters, market makers, liquidity providers, treasury operators, clearing systems, collateral platforms, or tokenization infrastructure. But it should be recognized only when the asset must be pre-positioned, reserved, collateralized, inventoried, or buffered for operational reasons.

The next section develops this location-based retained-demand framework. It separates the type of retained demand from the identity of the holder and explains why the holder question becomes central after user abstraction.

3. Backend Retained Demand

The retained-demand framework distinguishes visible usage from economically meaningful holding. In that framework, retained demand may take several forms: inventory demand, collateral demand, liquidity-buffer demand, and waiting liquidity. The present paper does not redefine those categories. Instead, it adds a second dimension: the location of retained demand.

This distinction becomes necessary after user abstraction. If users no longer hold the asset directly, the retained-demand question does not disappear. It becomes more specific. One must ask whether the asset is held elsewhere in the system, and if so, by whom, for what purpose, and under what operational constraints.

Backend retained demand refers to retained demand located at the infrastructure layer rather than the end-user layer. It may appear on the balance sheet, custody structure, liquidity system, collateral framework, operational inventory, or settlement account of an intermediary or infrastructure provider. The relevant holder may be a wallet operator, application operator, exchange, broker, custodian, paymaster, market maker, liquidity provider, treasury operator, clearing system, collateral platform, or tokenization infrastructure provider.

The important point is that backend retained demand is not established merely because a backend actor exists. A wallet may support an asset without retaining meaningful balances. A custodian may be able to hold an asset without that asset being operationally necessary. A market maker may quote an asset without maintaining significant inventory across time. A paymaster may sponsor fees while relying on rapid replenishment rather than material pre-positioned balances. A platform may route through an asset without becoming balance-sheet dependent on it.

For that reason, backend retained demand must be distinguished from backend capability. Capability means that an operator can use, support, custody, route, or process the asset. Retained demand means that the operator must maintain the asset in held form because not doing so would introduce cost, delay, fragility, market

impact, settlement risk, collateral failure, or operational unreliability. The first is infrastructure readiness. The second is balance-sheet or operational necessity.

Customer Utility Does Not Imply Asset Necessity

Customer utility is not asset necessity. A treasury, payments, or custody platform may deliver substantial customer value through fiat rails, stablecoins, internal netting, custody connectivity, and automated workflows without generating retained demand for a specific bridge asset. The relevant audit question is therefore not whether customers benefit from an infrastructure stack, but whether removing a specific asset from that stack materially worsens cost, latency, liquidity availability, settlement reliability, collateral efficiency, cross-ledger reach, or customer outcomes. Where customer outcomes remain unchanged without the asset, retained demand is not established.

This distinction connects directly to the removal-sensitivity gate developed in the preceding paper: customer utility establishes infrastructure value, not asset necessity, and only removal-sensitive degradation can establish the latter.

Operator-Layer Cost Compression

Customer utility may be asset-independent at the interface layer while remaining asset-dependent at the operator-cost layer. Even where customer-facing outcomes appear asset-independent, a specific asset may still matter at the operator layer if it allows equivalent customer utility to be delivered with materially lower prefunding requirements, liquidity fragmentation, hedging cost, settlement complexity, or operational complexity. The relevant audit question becomes whether the asset-inclusive configuration delivers the same customer outcome at lower total operator cost than the asset-free configuration. Such a claim should not be inferred from architecture alone; it requires comparative evidence that the asset-inclusive configuration reduces total operator cost or risk relative to viable asset-free alternatives.

Layer	What it means	Audit status
Capability	The platform can support, hold, route, custody, process, or display the asset.	Insufficient: capability does not imply operational necessity.
Customer utility	Customers gain efficiency, visibility, automation, treasury control, or payment functionality.	Insufficient: customer value may be delivered without the asset.
Retained demand / asset necessity	Operators must pre-position, collateralize, buffer, inventory, or otherwise hold the asset because removal worsens outcomes.	Required before backend retained demand is claimed.

A retained-demand claim is therefore incomplete unless it identifies the actor that carries the exposure. In user-facing digital asset narratives, demand is often implicitly located in the end user. In abstracted infrastructure environments, that assumption no longer holds. Demand may relocate to infrastructure-layer actors, or it may fail to form at all. The analyst must identify which outcome is occurring.

A useful backend retained-demand audit therefore requires two questions to be asked together. First, what form of retained demand is alleged? Second, whose balance sheet or operational system carries it?

The following table provides a location-based classification of possible backend retained-demand holders.

Holder type	Possible retained-demand form	Typical reason for holding	Key falsifier
End user	Direct holding, transactional balance, speculative or functional balance	User directly needs or chooses the asset	Exposure is fully abstracted and no end-user balance is required
Wallet or app operator	Gas inventory, payment inventory, routing inventory	Seamless execution, fee sponsorship, routing reliability	Operator can procure just in time or outsource exposure
Exchange or broker	Trading inventory, withdrawal inventory, settlement balance	Client execution, withdrawals, internal settlement	Internal netting or external liquidity removes need for held balances
Custodian	Custody balances, collateral support, omnibus balances	Safekeeping, institutional access, collateral administration	Custody support remains passive without operational use

Holder type	Possible retained-demand form	Typical reason for holding	Key falsifier
Market maker or LP	Execution inventory, AMM/DEX depth, corridor liquidity	Spread control, execution quality, slippage reduction	Quoting is possible without sustained inventory
Paymaster	Gas-token inventory, fee reserve, replenishment balance	Gasless user experience and transaction reliability	Dynamic replenishment is sufficient
Treasury operator	Liquidity buffer, settlement reserve, working balance	Treasury continuity, timing mismatch, liquidity readiness	Treasury optimization minimizes or eliminates asset-specific balances
Clearing or settlement operator	Waiting liquidity, settlement inventory, liquidity buffer	Settlement timing, fail prevention, cross-domain execution	Atomic settlement or internal ledgering removes waiting liquidity
Collateral or margin platform	Posted collateral, margin inventory, pledged balances	Eligibility, margining, financing, risk support	Eligibility exists but actual posting does not occur
RWA or tokenization platform	Settlement asset, collateral mobility asset, liquidity bridge	Tokenized-asset settlement or cross-asset mobility	Stablecoins, deposits, or internal settlement absorb the workflow

This table is not intended to imply that all listed actors generate retained demand. It identifies where retained demand could appear if operational conditions require it. Each row must be tested against evidence. The presence of a holder type is not sufficient. The relevant question is whether the holder must maintain the asset in retained form.

The correspondence between holder locations and retained-demand types should be read as an orthogonal mapping rather than a one-to-one classification. The location framework also helps connect the present paper back to the retained-demand categories introduced earlier in the series.[3] Market makers and liquidity providers are the clearest potential locations for inventory demand. Collateral platforms, custodians, and margin systems are the likely locations for collateral demand. Treasury operators may carry liquidity-buffer demand. Clearing, settlement, and cross-domain execution systems are the most natural locations for waiting liquidity. The forms of demand remain the same; what changes is the analytical need to locate the holder.

This distinction also clarifies the relationship between backend retained demand and institutional adoption. Institutional support may be an enabling condition, but it is not the same thing as retained demand. A custodian listing an asset may make institutional holding possible. A prime broker supporting an asset may make execution easier. A wallet integrating an asset may make user access smoother. But none of these developments proves that the asset is being held as inventory, collateral, buffer, or waiting liquidity.

Backend retained demand is stronger when holding is workflow-linked. A balance held for speculative reasons may matter economically, but it does not establish operational retained demand. A passive custody balance may show ownership, but not necessarily functional necessity. By contrast, an asset retained because it supports execution, settlement reliability, margin, liquidity buffering, or time-critical routing is closer to the concept developed here.

Three features are especially relevant. First, the holding must be connected to a workflow. Second, the holding must persist beyond momentary use. Third, the holding must reduce a cost or risk that would otherwise arise. If these elements cannot be identified, the claim should remain provisional.

This location-based approach also prevents a common overreading of user abstraction. When a user no longer holds the asset, it is tempting to assume that the platform must be holding it instead. Sometimes this may be true. But in many systems, abstraction is achieved precisely by minimizing asset-specific balance-sheet exposure. The backend may net flows internally, outsource liquidity, rely on stablecoins, route through existing settlement assets, or procure tokens only when required.

For that reason, the next section introduces the central test of this paper: the distinction between pre-positioned inventory and just-in-time liquidity. Backend retained demand becomes meaningful only when just-in-time procurement is insufficient or inferior to maintaining inventory in advance.

4. Pre-Positioned Inventory vs Just-in-Time Liquidity

The central audit distinction in this paper is between pre-positioned inventory and just-in-time liquidity. User abstraction may move asset exposure away from the end user, but it does not tell us whether an infrastructure-layer actor must hold the asset in advance. That question depends on whether the asset must be pre-positioned or can instead be procured only when needed.

Pre-positioned inventory refers to assets held before the moment of use in order to support execution, settlement, collateral, liquidity provision, fee sponsorship, market-making, routing, or operational resilience.[4] The asset is retained because waiting until the moment of need would be too slow, too costly, too risky, too uncertain, or institutionally impractical.

Just-in-time liquidity refers to the ability to acquire or mobilize the asset only when needed, use it briefly, and release, hedge, or dispose of the exposure without maintaining meaningful prior inventory. In this case, the asset may still be used, but the use does not create strong retained demand. The system depends on access, not possession.

Use without inventory is weak retained demand.

The proposition does not mean that transactional use is irrelevant. It means that use alone is not sufficient to establish retained demand. A digital asset can support a workflow while remaining balance-sheet light. It can be routed through, converted through, or paid through without being held in material form by users or infrastructure operators. If the asset can be procured on demand and released immediately, the retained-demand claim remains weak.

Backend retained demand becomes more likely when the expected cost and risk of just-in-time procurement exceed the cost and friction of maintaining pre-positioned inventory. The operational break-even condition can be stated simply: backend retained demand strengthens when expected JIT procurement risk plus execution cost exceeds the combined cost of capital, custody, governance, and institutional friction required to maintain controlled inventory. In practical terms, the boundary is reached when JIT procurement cost plus JIT failure risk exceeds inventory holding cost plus inventory friction.

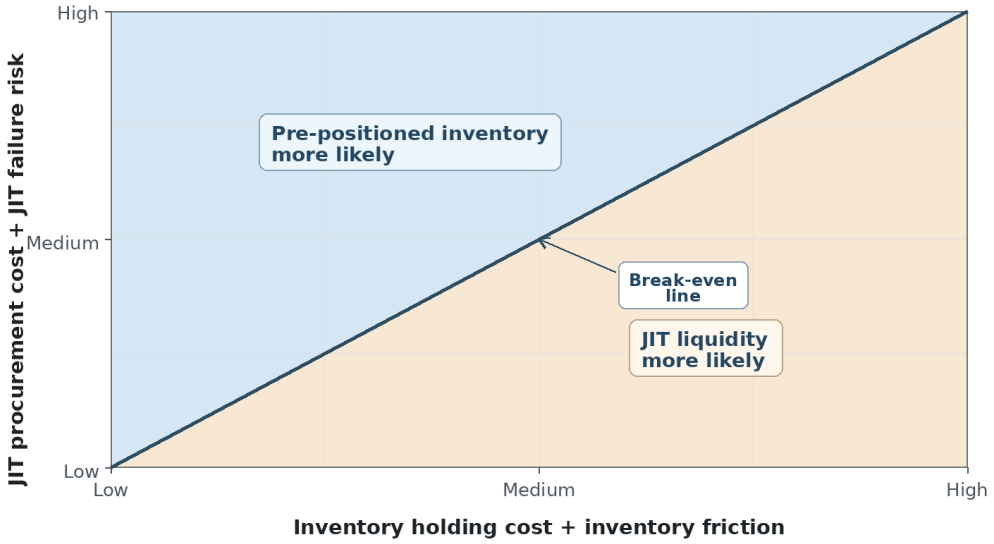

Figure 2. The JIT procurement versus pre-positioned inventory break-even line.

Dimension	Pre-positioned inventory	Just-in-time liquidity
Timing	Asset is held before use	Asset is acquired when needed
Balance-sheet exposure	Higher and more persistent	Lower and shorter-lived
Operational reliability	Stronger if execution must be guaranteed	Strong if procurement is reliable
Cost structure	Holding cost, custody cost, capital cost, risk cost	Procurement cost, spread, slippage, access risk
Retained-demand implication	Stronger if workflow-linked	Weaker if sufficient
Main risk	Inventory friction and asset exposure	Execution failure, market impact, counterparty failure

This comparison should not be treated as a binary choice in all cases. Many systems operate along a continuum. An operator may hold small working balances and replenish frequently. A market maker may maintain dynamic inventory targets. A paymaster may keep only enough gas inventory for short execution windows. A treasury system may combine minimum operational balances with external liquidity access. The relevant question is not whether inventory exists in an absolute sense, but whether the workflow requires meaningful pre-positioned exposure.

For that reason, inventory can be evaluated through density rather than mere presence. Inventory density may be approximated by the size of the balance, the duration of holding, the stability of the balance, the identity of the holder, the replenishment pattern, and the linkage to a specific workflow.

Just-in-time liquidity weakens retained-demand claims when the asset can be acquired quickly enough, without material market impact or slippage, from reliable counterparties and venues, and without regulatory, accounting, custody, or operational constraints that prevent timely procurement. If these conditions hold, backend operators may rationally avoid holding inventory.

JIT insufficiency condition	Why it matters
Execution window	The workflow requires execution faster than procurement can reliably occur
Market impact	On-demand acquisition moves price or creates unacceptable slippage
Counterparty risk	The required counterparty or liquidity source may not be available
Stress condition	Market disruption raises procurement cost or prevents access
Reliability / UX	User or client experience requires guaranteed execution
Collateral / margin lock	Assets must be posted or locked before use
Regulatory / operational constraint	Approval, custody, accounting, or compliance rules prevent instant procurement

These conditions are especially important because they convert use into potential retained demand. If a paymaster must guarantee transaction execution, it may need gas inventory. If a market maker must quote institutional-size trades, it may need inventory to manage slippage and response time. If a collateral platform recognizes an asset for margin, it may require posted balances rather than just-in-time access. If a settlement operator faces time-critical obligations, it may need waiting liquidity. If stress conditions make liquidity unreliable, pre-positioning may become a resilience requirement.

The execution window is often the first constraint. In many digital workflows, the relevant time interval may be too short for external procurement. Market impact is the second constraint: just-in-time procurement may work for small amounts but fail for larger or more concentrated flows. Counterparty risk is the third constraint, because just-in-time liquidity assumes that someone is available to provide the asset when needed. Stress conditions amplify all three constraints.

Reliability and user experience create another route to inventory. A consumer application, payment platform, or embedded wallet may not tolerate failed transactions, delayed settlement, or unpredictable fees. Even if just-in-time procurement is theoretically possible, the platform may hold working balances to preserve smooth user experience.

Collateral and margin create an even stronger distinction. A collateral asset cannot usually be procured only after collateral is needed. It must be posted, pledged, locked, or otherwise recognized in advance. Unlike other JIT insufficiency conditions, which reflect practical constraints of speed, cost, or counterparty availability, collateral and margin requirements are structurally incompatible with just-in-time procurement: an asset cannot be posted as collateral after it is already needed.[5]

Regulatory and operational constraints may also make just-in-time procurement impractical. A regulated institution may need pre-approved custody arrangements, internal risk limits, accounting treatment, compliance screening, governance approval, or counterparty onboarding before it can acquire and use an asset. In such cases, operational rules can make inventory necessary or can prevent the asset from being used at all.

The JIT-inventory boundary is therefore the core audit line. A retained-demand thesis strengthens when evidence shows that operators maintain assets because just-in-time procurement is too risky, too slow, too costly, or institutionally impractical. It weakens when evidence shows that usage occurs without meaningful inventory, collateral, buffer, or waiting-liquidity balances.

This paper therefore treats just-in-time liquidity as a falsification condition. It does not deny that assets can be useful without inventory. It denies that use without inventory is sufficient to establish retained demand. The audit standard is stricter: who must hold the asset, why must they hold it, how long must they hold it, and what fails if they do not?

5. Operator Friction and Multi-Asset Management

This section does not re-examine institutional admissibility, which was addressed in the preceding series. Its narrower purpose is to show how operator friction shifts the break-even point between just-in-time procurement and pre-positioned inventory.[6] In this paper, friction is not a new gate; it is a boundary-shifter.

The distinction between pre-positioned inventory and just-in-time liquidity cannot be evaluated only through market liquidity. It must also be evaluated through operator friction. Backend retained demand appears inside real operating entities, and those entities face constraints that are not visible from the user interface or from transaction flow alone.

Operator friction refers to the cost, complexity, risk, and governance burden associated with holding, managing, and relying on a digital asset inside an operational workflow. It includes custody arrangements, key management, accounting treatment, tax handling, regulatory classification, compliance monitoring, risk limits, internal approvals, auditability, liquidity management, hedging, balance-sheet volatility, counterparty onboarding, and operational continuity requirements.

The relevant question here is not simply whether an asset is institutionally admissible. The question is whether operator friction shifts the boundary between just-in-time procurement and pre-positioned inventory. If holding the asset is more costly than procuring it when needed, operators will tend to minimize inventory. If procuring the asset when needed is riskier or more expensive than holding it in advance, operators may maintain inventory despite the friction.

Operator friction therefore cuts in both directions. On one side, friction discourages retained demand because each additional asset increases custody complexity, accounting burden, risk monitoring, approval requirements, and operational controls. On the other side, friction can make controlled inventory more rational than repeated ad hoc procurement when the workflow requires reliable execution, pre-approved compliance treatment, defined risk limits, or guaranteed collateral availability.

The practical question is therefore comparative: does the cost and friction of maintaining inventory exceed the expected cost and risk of procuring the asset just in time? This comparison may change by asset, holder type, jurisdiction, workflow, market condition, and stress regime.

Friction type	Effect on JIT vs inventory
Custody and key management	High custody burden discourages inventory; reliable custody can make controlled balances feasible
Accounting / tax treatment	Difficult reporting discourages holding; standardized treatment can support inventory
Regulatory classification	Unclear status discourages retention; clarity can move an asset from capability to inventory
Compliance operations	AML, sanctions, and monitoring can slow JIT procurement or require pre-approved flows
Internal governance and risk limits	Committees, exposure limits, and policy controls may prevent ad hoc procurement
Liquidity management	Fragmented liquidity makes JIT riskier; deep reliable markets make JIT easier
Auditability	Traceable controlled balances may be easier to audit than repeated emergency procurement

This table should not be read as a list of reasons why operators will hold more assets. In many cases, it explains why they will hold fewer. Multi-asset management is costly. Each additional asset introduces new operational surfaces: custody accounts, transfer controls, monitoring rules, treasury policies, valuation procedures, liquidity venues, counterparty lists, disaster recovery plans, and escalation procedures.

However, standardization pressure should not be confused with inevitable convergence. Operators may prefer fewer assets, but they may still maintain multiple assets or rails when redundancy, jurisdictional coverage, liquidity access, collateral eligibility, or workflow specificity requires it.

For that reason, operator friction does not produce a simple prediction. It produces a boundary test. When friction dominates, operators tend to avoid inventory and rely on abstraction, netting, stablecoins, tokenized deposits, or just-in-time procurement. When execution risk, timing risk, collateral lockup, stress risk, or reliability requirements dominate, operators may hold inventory despite friction.

The same logic explains why backend retained demand should not be inferred from technical support. A platform may support an asset but avoid holding it. A custodian may be able to safeguard an asset but not use it operationally. A wallet may integrate an asset while relying on external liquidity. The operational question is not support, but retained exposure.

Operator friction therefore refines the central test of the paper. Backend retained demand is not created by user abstraction alone. It emerges when an infrastructure-layer actor rationally accepts the cost and friction of holding an asset because the alternative - just-in-time procurement, substitution, netting, or outsourcing - would create greater operational risk, cost, delay, or fragility.

6. Cross-Asset Application and Evidence Status

The preceding sections developed a location-based retained-demand framework and the distinction between pre-positioned inventory and just-in-time liquidity. This section applies that framework across several major asset categories. The purpose is not to rank assets, forecast prices, or declare which asset will become indispensable. It is to show how the same audit questions can be applied across different digital-asset roles.

A cross-asset approach is necessary because user abstraction does not affect all assets in the same way. For some assets, user abstraction may reduce direct user holding while leaving backend holdings structurally strong. For others, abstraction may expose the fact that usage does not require meaningful asset retention. Some assets are more naturally held as reserves, collateral, gas inventory, or staking collateral. Others may be used in workflows without being held by end users or backend operators in significant form.

The table should be read as an audit map rather than a ranking of asset quality. Its purpose is to locate possible backend holders, retained-demand forms, JIT substitution risks, and falsification conditions across different asset categories.

Cross-Asset Audit Map

Asset / category	User abstraction pattern	Likely backend holder	Possible retained-demand form	JIT risk	Key falsifier	Evidence status
BTC	Users may access BTC through ETFs, custodians, exchanges, brokers, or treasury platforms without self-custody	Funds, custodians, exchanges, corporate treasuries, collateral desks	Reserve holding, custody balances, collateral, treasury allocation	Generally lower for long-duration reserve roles; higher for collateral or settlement under stress	BTC exposure becomes purely synthetic or derivative-based without underlying custody or collateral need	Established for reserve/custody exposure; active or contested for broader collateral and settlement roles
ETH / SOL	Gas fees may be abstracted through paymasters, embedded wallets, sponsored transactions, relayers, or app-level fee management	Wallets, apps, paymasters, validators, exchanges, staking providers, infrastructure operators	Gas inventory, staking/security participation, fee reserves, validator-related holdings, app working balances	Medium: JIT may work for small flows, but reliability, execution windows, and staking/security roles can require working inventory or locked exposure	Gasless UX scales while operators maintain only minimal dynamic balances and avoid meaningful inventory	Established for staking/security roles; active for gas inventory; workflow-dependent for paymaster inventory
LINK	End users rarely need to hold LINK to consume oracle-enabled applications	Oracle node operators, staking participants, infrastructure providers, service protocols	Staking/security collateral, oracle-service participation, possible operator reserves	Medium to high depending on whether service participation requires staking, locked exposure, or durable operator commitment	Oracle usage grows while LINK holding, staking, or collateral requirements remain weak, optional, or easily outsourced	Active / contested: oracle network use is visible, but retained-demand strength depends on staking design and operator economics
Stablecoins / tokenized deposits	Users may hold balances through apps, exchanges, payment platforms, wallets, treasury systems, or bank-like interfaces	Exchanges, payment firms, custodians, treasury operators, issuers, settlement platforms	Settlement balances, treasury balances, working capital, payment inventory, liquidity buffers	Lower for common settlement use if deep liquidity exists; higher under redemption, cross-venue, cross-border, or stress conditions	Stablecoin use remains pass-through with no meaningful operator balances beyond client custody or issuance backing	Established for settlement and treasury balances; role varies by issuer, jurisdiction, redemption structure, and backing design
XRP	Users may not hold XRP directly if payments, liquidity routing, or settlement are abstracted through providers	Market makers, liquidity providers, exchanges, brokers, custodians, payment or settlement operators, possibly treasury or collateral platforms	Bridge inventory, execution inventory, liquidity buffers, waiting liquidity, possible collateral or margin balances	High unless pre-positioning is required by execution window, slippage, stress liquidity, collateral, or cross-ledger routing	XRP-adjacent workflows scale through stablecoins, fiat rails, internal ledgers, or JIT liquidity without retained XRP inventory	Candidate / contested: structurally possible in some scenarios, but retained backend demand must be evidenced
XDC / RWA-specific assets	Users may access tokenized trade finance or RWA products without holding the network or settlement asset directly	RWA platforms, custodians, settlement operators, liquidity providers, validators, institutional service providers	Settlement inventory, collateral mobility asset, network participation, workflow-specific liquidity	Medium to high depending on whether RWA workflows require native asset retention or can settle in stablecoins/deposits	RWA adoption scales while settlement, collateral, and liquidity functions are absorbed by stablecoins, deposits, or platform ledgers	Candidate / unproven: role depends on actual RWA settlement design and operator balance-sheet behavior
Other L1 / L2 gas assets	Gas may be hidden through account abstraction, sponsored transactions, sequencer systems, relayers, or wallet-level routing	Sequencers, relayers, paymasters, apps, exchanges, validators, infrastructure providers	Gas inventory, staking/security holdings, fee reserves, sequencer or validator balances	Medium: small-user flows may be JIT, but large apps and infrastructure may need working balances	User activity grows while gas exposure is fully minimized through sponsorship, batching, third-party abstraction, or alternative fee systems	Active but workflow-dependent

This table is deliberately conservative. It does not infer retained demand from usefulness, visibility, or association. Each asset is evaluated according to where backend holding might appear and what would weaken that claim.

Several implications follow. BTC provides an anchor for long-duration held exposure: its strongest retained-demand evidence appears through custody, reserve allocation, fund structures, and possible collateral treatment rather than ordinary transactional usage. ETH and SOL provide the clearest user-abstraction examples: gasless UX and paymasters can remove the gas token from the user interface, but the audit question becomes whether backend actors must maintain working balances or can dynamically source and replenish the gas asset.

LINK illustrates the difference between service dependency and token retention. Oracle-enabled applications may rely on oracle services without end users holding LINK. The relevant question is whether oracle operators, staking participants, or service providers must hold, stake, reserve, or collateralize LINK in a way that creates durable backend demand.

Stablecoins and tokenized deposits provide a strong comparison case because they can be both retained-demand assets and bypass mechanisms for other assets.[7] Payment firms, exchanges, treasuries, and settlement platforms may maintain stablecoin balances, while stablecoins and tokenized deposits may also absorb settlement, treasury, and liquidity functions that might otherwise have required another asset.

XRP should be treated as a cross-asset candidate rather than the center of the framework. Its potential backend retained-demand channels include market-making inventory, bridge liquidity, cross-ledger settlement, waiting liquidity, liquidity buffers, and possible collateral or margin use. The retained-demand claim strengthens only if specific backend actors must hold XRP because just-in-time procurement, stablecoin routing, fiat rails, internal ledgers, or other substitutes are insufficient.

XDC and RWA-specific assets show why tokenized-asset infrastructure must be evaluated carefully.[8] RWA adoption may create settlement, collateral, and liquidity needs, but those needs may be satisfied through stablecoins, tokenized deposits, platform-specific ledgers, or conventional settlement arrangements. The relevant audit question is not whether RWA infrastructure grows, but whether settlement operators, custodians, or liquidity providers must maintain asset-specific balances that cannot be replaced by stablecoins, tokenized deposits, or platform-level accounting.

The evidence status column should be read as a classification of current retained-demand observability, not as a judgment of investment quality.

Evidence status	Meaning
Established	Retained holding, custody, reserve, collateral, staking, or operational balances are relatively observable
Active / observable	Usage and relevant operators are visible, but retained-demand strength varies by workflow
Candidate	The structure could create backend retained demand, but evidence remains limited or conditional
Contested	The relationship between usage and retained holding depends strongly on design, incentives, or institutional rules
Unproven	The retained-demand claim remains primarily theoretical or weakly evidenced

These statuses are intentionally distinct from asset quality or market capitalization. A large and important asset may still have weak evidence for a specific retained-demand channel. A smaller asset may have a narrow but clearer retained-demand role. The purpose of the classification is to prevent cross-asset analysis from becoming a ranking exercise.

This is the broader value of the backend retained-demand framework. It does not ask whether a token is visible to the user. It asks whether the system contains an actor that must hold the token. Gasless UX is not demand destruction or demand creation by itself. Oracle adoption is not token demand by itself. Tokenization is not settlement-asset demand by itself. Stablecoin growth is not automatically positive or negative for other assets.

Cross-ledger infrastructure is not bridge-asset demand by itself. Each claim depends on backend holder location, retained-demand form, and JIT feasibility.

This framework therefore shifts cross-asset comparison from narrative category to operational burden. The relevant question is not which asset is most discussed, most integrated, or most associated with institutional themes. The relevant question is which workflows require which actors to hold which assets, for how long, under what risks, and with what consequence if they do not.

7. Falsification Conditions

A backend retained-demand framework must be able to fail.[9] If the framework merely identifies possible holders, possible workflows, and possible retained-demand channels, it risks becoming another form of asset-adjacent narrative. The purpose of the framework is not to preserve every asset as a future candidate. It is to distinguish conditions under which backend retained demand exists from conditions under which usage, support, or abstraction occurs without meaningful retained holding.

The falsification logic follows directly from the central proposition. If an asset can be used without meaningful inventory, collateral, liquidity buffer, or waiting-liquidity balances, then retained demand remains weak. The most important falsifier is not that an asset is unused. The more subtle falsifier is that the asset is used but not retained.

The main falsification conditions can be grouped into three categories: procurement and substitution falsifiers, holder and balance-sheet falsifiers, and workflow or institutional falsifiers.

7.1 Procurement and Substitution Falsifiers

The first group concerns cases in which access replaces possession. The primary falsification condition is just-in-time sufficiency. If an asset can be reliably procured when needed, used immediately, and released without meaningful timing risk, market impact, counterparty risk, collateral constraint, or operational fragility, then backend retained demand remains weak.

Related falsifiers include substitution by stablecoins, tokenized deposits, fiat rails, or other settlement assets, and internal netting or closed-loop absorption. If payment, settlement, liquidity, or RWA workflows scale through substitutes, or if platforms offset flows internally and settle only residual exposures externally, visible user activity may grow while asset-specific backend retained demand remains modest.

7.2 Holder and Balance-Sheet Falsifiers

The second group concerns failure to locate the holder of retained demand. A retained-demand claim is incomplete if it cannot identify the actor that carries the exposure. If the claim asserts that an asset is needed but cannot specify whether the holder is a market maker, paymaster, custodian, exchange, treasury operator, settlement layer, collateral platform, or liquidity provider, then the claim remains structurally underdeveloped.

Backend capability without retained exposure is a related condition. Custody support, wallet integration, exchange listing, bridge compatibility, paymaster support, or treasury-system readiness may indicate capability, but not retained demand. Outsourcing also requires careful treatment: if a visible operator shifts exposure to downstream liquidity providers, the audit must relocate the holder rather than assume the visible operator carries the demand.

7.3 Workflow and Institutional Falsifiers

The third group concerns workflows that fail to create retained balances even when the asset appears technically relevant. Collateral eligibility without actual posting is one example. Eligibility is not use. If an asset is eligible as collateral but is not actually posted in meaningful amounts, or if haircuts, concentration limits, volatility treatment, or governance constraints make it unattractive, then the collateral retained-demand claim remains weak.

Operator friction can also prevent inventory formation. In some cases, just-in-time procurement may be insufficient in theory, but regulatory uncertainty, custody burden, accounting treatment, internal governance, volatility limits, capital constraints, or compliance requirements may still lead operators to avoid both inventory and the workflow itself. Similarly, system design may reduce waiting-liquidity demand if atomic settlement, synchronized ledgers, internal clearing, real-time settlement, or liquidity-saving mechanisms compress or eliminate temporal gaps.

Customer value alone does not confirm retained demand; only removal-sensitive degradation can establish asset necessity.

Operator-layer cost compression should not be inferred from interface abstraction or customer utility alone; it requires comparative evidence that the asset-inclusive configuration reduces total operator cost or risk relative to viable asset-free alternatives.

Falsification condition	Retained-demand implication
JIT procurement is sufficient	Access replaces inventory; retained demand remains weak
Stablecoins / deposits / fiat rails substitute	Other settlement assets absorb the workflow
Internal netting absorbs flows	User activity grows without asset-specific balances
Backend holder cannot be identified	Demand claim cannot be located or audited
Capability exists without retained exposure	Support does not become balance-sheet necessity
Customer value without asset necessity	Customers benefit from the infrastructure stack, but removal of the asset does not materially worsen outcomes
Exposure is fully outsourced	Holder may be elsewhere, or retained demand may be absent
Collateral eligibility lacks actual posting	Collateral demand remains theoretical
Operator friction blocks retention	Structural possibility fails institutional adoption
Atomic settlement reduces timing gaps	Waiting liquidity demand declines
Paymaster model uses minimal dynamic balances	Gas abstraction does not imply strong inventory
Market-making occurs without durable inventory	Liquidity visibility does not prove retained holding
Usage grows without backend inventory	Abstraction improves UX without retained demand

The table preserves a broader checklist, but the main analytical burden rests on the three categories above: procurement and substitution, holder and balance-sheet location, and workflow or institutional failure. The framework is conservative by design. It does not treat every abstracted workflow as evidence of hidden demand. It asks whether a specific actor must hold a specific asset in a specific form for a specific operational reason. If that chain cannot be established, the retained-demand claim remains provisional.

In institutionally connected digital asset infrastructure, use may become increasingly abstracted, routed, netted, outsourced, or momentary. The analytical risk is to mistake invisible usage for invisible retained demand. The framework developed here is intended to prevent that error.

8. Conclusion

This paper has argued that user abstraction changes the location of the retained-demand question. As digital asset workflows become more embedded, automated, custodial, and user-friendly, token exposure may disappear from the user interface. Users may transact without holding gas tokens, interact with applications through embedded wallets, access digital assets through custodians or platforms, or participate in tokenized financial workflows without observing the underlying settlement mechanics.

That disappearance is analytically ambiguous. It may mean that the asset has become less relevant to the user. It may also mean that the asset has moved deeper into the infrastructure layer. The central question is therefore not whether the user holds the token. The question is whether someone else must hold it.

The paper developed backend retained demand as a location-based extension of retained-demand analysis. The prior retained-demand framework distinguished usage from inventory demand, collateral demand, liquidity-buffer demand, and waiting liquidity. The present paper asked where those forms of demand appear after user-facing token exposure is abstracted away. The answer cannot be assumed. It must be audited at the level of

backend holders: wallets, applications, exchanges, brokers, custodians, paymasters, market makers, liquidity providers, treasury operators, settlement systems, collateral platforms, and tokenization infrastructure.

The paper's central distinction is between pre-positioned inventory and just-in-time liquidity. Pre-positioned inventory indicates that an actor holds the asset before use because execution, settlement, collateral, liquidity provision, routing, fee sponsorship, or resilience requires it. Just-in-time liquidity indicates that the asset can be acquired only when needed, used briefly, and released without meaningful prior inventory. The difference is decisive because use alone does not establish retained demand.

The central proposition can therefore be stated simply: use without inventory is weak retained demand.

This proposition is not a rejection of digital asset utility. It is a stricter audit standard. An asset may be useful, technically necessary, integrated, or widely routed without becoming meaningfully retained. Backend retained demand becomes stronger only when just-in-time procurement is too slow, too costly, too risky, too fragile, or institutionally impractical relative to maintaining inventory.

The cross-asset application showed why this distinction matters beyond any single asset. BTC may display retained demand through custody, reserve allocation, and collateral-like roles. ETH and SOL illustrate user abstraction through paymasters, gasless UX, and infrastructure-managed gas exposure. LINK shows the distinction between service adoption and token-retention pressure. Stablecoins and tokenized deposits may generate strong settlement and treasury balances while also absorbing workflows that might otherwise require other assets. XRP remains a candidate for backend retained demand in market-making, bridge liquidity, cross-ledger settlement, waiting liquidity, or collateral contexts, but only if specific actors must hold it.

The framework is intentionally non-promotional and non-dismissive. It does not assume that user abstraction destroys asset demand. It also does not assume that hidden backend demand must exist. Instead, it asks a practical sequence of questions: who holds the asset, in what form, for what workflow, for how long, under what constraints, and what fails if they do not?

This sequence is useful for infrastructure designers, custodians, paymasters, exchanges, market makers, treasury operators, settlement platforms, and analysts because it translates abstract asset narratives into operational balance-sheet questions. It asks whether a workflow creates inventory, collateral, buffer, or waiting-liquidity demand, or whether it can operate through JIT procurement, netting, outsourcing, stablecoins, tokenized deposits, or internal ledgers.

The falsification conditions are therefore as important as the supportive conditions. If JIT procurement is sufficient, backend retained demand remains weak. If no backend holder can be identified, the claim cannot be located. If custody support remains passive, capability does not become demand. If stablecoins, tokenized deposits, or internal ledgers absorb the workflow, the asset-specific retained-demand claim weakens. If collateral eligibility does not lead to actual posting, collateral demand remains theoretical. If usage grows without backend inventory, user abstraction has not produced retained demand.

The broader contribution of this paper is methodological and operational. Retained demand after user abstraction should be audited at the infrastructure balance-sheet layer, not at the user interface. The disappearance of token exposure from the user screen is not the end of demand analysis. It is the beginning of a more precise audit.

In this sense, the paper opens a second phase of the retained-demand audit framework. The first phase asked whether retained demand exists, how it differs from usage, how it survives closure competition, how it can be translated into liquidity-density burden, and how evidence gates sizing. The second phase asks where that demand is carried after user abstraction, whose balance sheet absorbs it, and whether just-in-time liquidity prevents retained demand from forming.

For the series as a whole, this paper closes the move from retained-demand type to retained-demand location. It reinforces the central discipline of the framework: retained demand should not be inferred from usage,

adjacency, technical support, or abstraction alone. It must be located in a specific holder, a specific retained-demand form, a specific workflow, and a specific failure condition if that holding is absent.

Appendix A: Backend Retained Demand Audit Template

This appendix provides a practical audit template for applying the backend retained-demand framework developed in this paper. It is intended for use by analysts, infrastructure designers, market participants, custodians, paymasters, treasury operators, settlement platforms, and researchers evaluating whether a user-abstracted digital asset workflow creates retained demand.

The template is designed to function as a detachable worksheet.[10] It can be applied to a specific asset, workflow, platform, or infrastructure layer without requiring the reader to adopt any price thesis or asset-specific conclusion.

The template should be applied conservatively. It should not infer backend retained demand from user activity, technical support, wallet integration, custody availability, exchange listing, or ecosystem visibility alone. The relevant question is whether a specific actor must retain the asset in a specific form for a specific operational reason.

A.1 Core Audit Questions

Audit item	Question
Asset / token	What asset is being evaluated?
Workflow	What workflow uses or may use the asset?
User abstraction layer	What exposure is hidden from the user?
Customer utility	What customer value does the workflow deliver?
Asset necessity test	Would removing the asset materially worsen cost, latency, liquidity, settlement reliability, collateral efficiency, cross-ledger reach, or customer outcomes?
Backend holder	Who may need to hold the asset?
Holding form	Inventory, collateral, liquidity buffer, waiting liquidity, or other?
Operational reason	Why would the asset need to be held?
JIT feasibility	Can the asset be procured only when needed?
JIT failure condition	Under what condition would JIT fail?
Inventory friction	What cost or risk comes from holding inventory?
Procurement risk	What cost or risk comes from not holding inventory?
Evidence status	Established, active, candidate, contested, or unproven?
Falsifier	What evidence would weaken the retained-demand claim?

A.2 Backend Holder Classification

Holder type	Evidence to look for
Wallet / app operator	Gas reserves, fee sponsorship balances, routing balances, user-experience reliability buffers
Exchange / broker	Withdrawal inventory, trading inventory, internal settlement balances, client execution reserves
Custodian	Operational custody balances, collateral administration, pledged balances, client liquidity support
Market maker / LP	Durable inventory, AMM / DEX depth, spread control, corridor liquidity, stress-time liquidity
Paymaster	Gas-token working balances, replenishment patterns, failed-transaction prevention reserves
Treasury operator	Liquidity buffers, settlement balances, working capital, asset-specific reserves
Clearing / settlement operator	Waiting liquidity, settlement inventory, liquidity-saving residual balances
Collateral / margin platform	Posted collateral, margin balances, haircut schedules, concentration-limit treatment
RWA / tokenization platform	Settlement-asset balances, collateral mobility assets, redemption or liquidity reserves

A.3 Retained-Demand Form

Form	Audit question
Inventory demand	Is the asset held to support recurring execution, routing, settlement, or market-making?
Collateral demand	Is the asset posted, pledged, locked, or recognized as support for obligations?
Liquidity-buffer demand	Is the asset held to absorb uncertainty, stress, mismatch, or execution risk?
Waiting liquidity	Is the asset held across a time gap while settlement, conversion, or transfer is pending?

A.4 JIT Feasibility Test

Test	Question	If yes	If no
Speed	Can the asset be acquired fast enough?	JIT more plausible	Inventory more likely
Depth	Can it be acquired without material slippage?	JIT more plausible	Inventory more likely
Counterparty availability	Are venues and counterparties reliably available?	JIT more plausible	Inventory more likely
Stress resilience	Does access remain available under stress?	JIT more plausible	Inventory more likely
Operational approval	Can the operator acquire it instantly under internal rules?	JIT more plausible	Inventory more likely
Collateral timing	Can the asset be posted only when needed?	JIT more plausible	Pre-positioning more likely
UX reliability	Can failures or delays be tolerated?	JIT more plausible	Inventory more likely

A.5 Operator Friction Test

Friction type	Audit question
Custody	Can the operator safely custody the asset?
Key management	Are transfer controls and recovery procedures manageable?
Accounting	Can the asset be valued and reported without excessive burden?
Tax	Does holding or frequent disposal create tax/reporting complexity?
Regulatory	Is the asset permissible under relevant rules?
Compliance	Can AML, sanctions, and transaction-monitoring requirements be met?
Governance	Can internal committees approve the asset and workflow?
Risk limits	Can volatility, liquidity, and concentration risks fit within policy limits?
Hedging	Can inventory risk be hedged or offset?
Auditability	Can balances and movements be audited clearly?

A.6 Evidence Status Definitions

Status	Meaning
Established	Retained holding, custody, reserve, collateral, staking, or operational balances are relatively observable
Active / observable	Usage and relevant operators are visible, but retained-demand strength varies by workflow
Candidate	The structure could create backend retained demand, but evidence remains limited or conditional
Contested	The relationship between usage and retained holding depends strongly on design, incentives, or institutional rules
Unproven	The retained-demand claim remains primarily theoretical or weakly evidenced

A.7 Falsification Checklist

Falsifier	Meaning
JIT procurement is sufficient	Asset can be acquired only when needed without meaningful risk
No backend holder identified	The retained-demand claim cannot be located
Capability without retention	Support exists but no meaningful balances are held
Customer value without asset necessity	The workflow creates customer value, but removing the asset does not materially worsen outcomes
Outsourced exposure	Visible operator does not hold the asset; exposure is shifted elsewhere
No downstream inventory	Even outsourced providers do not maintain meaningful inventory
Internal netting	Flows are offset before asset-specific settlement is needed
Stablecoin / deposit substitution	Workflow is absorbed by stablecoins, tokenized deposits, or fiat rails
Atomic settlement	Waiting liquidity is reduced or eliminated
Collateral non-use	Eligibility exists but actual posting does not occur
Minimal dynamic balances	Paymaster or app holds only negligible replenishment balances
No durable market-making inventory	Liquidity is quoted without sustained retained exposure
Operator friction blocks holding	Asset cannot be practically retained under institutional constraints
Usage without inventory	Workflow scales but no actor holds meaningful asset inventory

A.8 Compact Audit Template

Audit item	Result
Asset / token	-
Workflow	-
User abstraction pattern	-
Backend holder	-
Retained-demand form	Inventory / collateral / buffer / waiting liquidity / other
JIT feasible?	Yes / No / Conditional
JIT failure condition	-
Inventory friction	Low / Medium / High
Procurement risk	Low / Medium / High
Operator friction	Low / Medium / High
Evidence status	Established / Active / Candidate / Contested / Unproven
Key falsifier	-
Provisional classification	Backend retained demand likely / possible / weak / not shown
Update condition	-

Footnotes

1. This paper builds on the retained-demand framework developed in Meta (2026a), which distinguished visible usage from inventory demand, collateral demand, liquidity-buffer demand, and waiting liquidity. It also follows the broader retained-demand audit sequence developed in Meta (2026b-e): expansion versus closure, conditional liquidity-density sizing, compression / bypass / amplification, and evidence-gated sizing eligibility. The present paper does not redefine those concepts; it extends them by asking where retained demand is located after user-facing exposure is abstracted away.

2. Account abstraction and paymaster designs provide a concrete example of user-facing token exposure being delegated to infrastructure-layer actors. ERC-4337 proposes account abstraction without consensus-layer protocol changes and introduces UserOperation objects and paymaster-related mechanisms. The present paper does not treat any specific implementation as evidence of retained demand; it uses the paymaster model as an example of how the holder question can move from the end user to backend infrastructure.

3. The location-based classification in this paper should be read as orthogonal to the type-based classification introduced in Meta (2026a). For example, market makers and liquidity providers may be relevant locations for inventory demand; collateral platforms, custodians, and margin systems may be relevant locations for collateral demand; treasury operators may carry liquidity-buffer demand; and clearing or settlement systems may be natural locations for waiting liquidity.

4. The terminology used here is consistent with payment and settlement infrastructure literature that treats liquidity, settlement timing, collateral, and operational resilience as central components of financial market infrastructure risk management. See CPSS-IOSCO (2012) and CPMI (2023).

5. This distinction is consistent with the broader liquidity-preparedness literature on margin and collateral calls, which emphasizes ex ante liquidity planning, collateral management, governance, and stress preparedness rather than purely reactive funding after an obligation arises.

6. The institutional admissibility problem is treated more directly in Meta (2026e), which introduced an institutional friction gate as part of an evidence-gated sizing eligibility protocol. In this paper, operator friction is used more narrowly to evaluate the boundary between just-in-time procurement and pre-positioned inventory.

7. Stablecoins and tokenized deposits are treated here not as identical instruments, but as settlement and treasury alternatives that may affect the retained-demand profile of other assets. For broader discussions of tokenised money, tokenized deposits, and tokenisation of assets, see BIS (2023), CPMI (2024), EBA (2024), and Bank of Japan (2024).

8. This paper uses tokenization and RWA infrastructure as an application domain rather than as evidence of any particular asset's retained demand. The relevant issue is whether tokenized-asset workflows create asset-specific inventory, collateral, buffer, or waiting-liquidity needs, or whether settlement and accounting are absorbed by stablecoins, tokenized deposits, internal ledgers, or platform-level systems.

9. The falsification structure in this paper is consistent with the evidence-gated approach in Meta (2026e). However, the present paper applies that discipline to backend retained-demand claims rather than to downstream sizing eligibility.

10. The appendix is intended as an operational audit aid rather than a scoring model. It should not be interpreted as producing price forecasts, asset rankings, or investment conclusions.

Keywords

User abstraction; backend retained demand; backend inventory formation; pre-positioned inventory; just-in-time liquidity; customer utility; asset necessity; removal sensitivity; digital assets; retained demand; inventory demand; collateral demand; liquidity buffers; waiting liquidity; infrastructure balance sheets; operator friction; institutional settlement; paymasters; stablecoins; tokenized deposits; XRP; LINK; BTC; ETH; SOL; XDC; RWA; market-making; operational audit; financial market infrastructure

Author Note

This paper is presented as a working paper and should be read as a framework-oriented contribution. It is the sixth paper in a retained-demand audit series for institutionally connected digital assets. Its purpose is to clarify how user abstraction changes the location of retained-demand analysis and how just-in-time liquidity may falsify backend retained-demand claims.

The paper does not provide investment advice, a price target, or a recommendation to buy, sell, or hold any digital asset. XRP appears only as one cross-asset case in a broader audit framework.

The contribution of the paper is methodological and operational: it asks whether retained demand appears on backend balance sheets or operational systems after user-facing token exposure disappears, and whether operators must pre-position, reserve, collateralize, inventory, or buffer an asset rather than procure it just in time.

Short Description

A working paper extending retained-demand analysis from demand type to demand location after user abstraction. The paper distinguishes backend retained demand from backend capability, customer utility from asset necessity, and pre-positioned inventory from just-in-time liquidity, and provides a cross-asset audit map for evaluating whether backend operators must hold assets in inventory, collateral, liquidity-buffer, or waiting-liquidity form.

References

Bank for International Settlements. 2023. Annual Economic Report 2023: Blueprint for the Future Monetary System. Bank for International Settlements.

Bank of Japan. 2024. Deposit Tokenization. Bank of Japan Review.

Basel Committee on Banking Supervision. 2013. Basel III: The Liquidity Coverage Ratio and Liquidity Risk Monitoring Tools. Bank for International Settlements.

Committee on Payment and Settlement Systems and Technical Committee of the International Organization of Securities Commissions. 2012. Principles for Financial Market Infrastructures. Bank for International Settlements and IOSCO.

Committee on Payments and Market Infrastructures. 2023. A Glossary of Terms Used in Payments and Settlement Systems. Bank for International Settlements.

Committee on Payments and Market Infrastructures. 2024. Tokenisation in the Context of Money and Other Assets. Bank for International Settlements.

Ethereum Improvement Proposals. 2021. ERC-4337: Account Abstraction Using Alt Mempool. Ethereum Improvement Proposals.

European Banking Authority. 2024. Report on Tokenised Deposits. European Banking Authority.

Financial Stability Board. 2024. Liquidity Preparedness for Margin and Collateral Calls. Financial Stability Board.

Meta, S. 2026a. From Usage to Retained Demand: A Structural Audit Framework for Institutionally Connected Digital Assets. Working Paper.

Meta, S. 2026b. Expansion Is Not Closure: Settlement Stack Competition and the Conditional Relevance of External Connective Assets. Working Paper.

Meta, S. 2026c. From Retained Demand to Required Liquidity Density: A Conditional Sizing Framework for Institutionally Connected Digital Assets. Working Paper.

Meta, S. 2026d. Compression, Bypass, and Amplification: An Applied Stack-Audit Framework for Institutionally Connected Digital Assets. Working Paper.

Meta, S. 2026e. Evidence Before Sizing: An Operational Audit Protocol for XRP-Adjacent Institutional Infrastructure. Working Paper.